### 4.1 Datatypes

LOTUS' data model consists of tables with structured and unstructured fields (e.g., text or images). LOTUS' semantic operators can take both of these data-types as parameters to a langex. Additionally, LOTUS supports *semantic indices* over natural-language text columns to provide optimized query processing. These indices leverage semantic embeddings over each document in the column and capture semantic similarity using embedding distance metrics. Semantic indices can be created off-line with `sem_index` and a specified retriever model, and then loaded from disk using `load_sem_index`.

### 4.2 Semantic Operators in LOTUS

We now overview the semantic operators supported in the LOTUS API, which includes the core set of operators described in Section 2, as well as several additional variants provided for convenience. Each operator takes optional parameters to specify a target accuracy and error probability, which the optimizer will use to transparently perform optimizations.

***Sem_filter, Sem_join, & Sem_sim_join.*** The LOTUS API supports `sem_filter` and `sem_join`, both of which take a langex predicate, as described in Section 2. In addition, LOTUS provides a join variant, `sem_sim_join`, where tuples are matched according to their *semantic similarity*, rather than an arbitrary natural-language predicate. Akin to an equi-join in standard relational algebra, the semantic similarity join is a specialized semantic join, which indicates additional optimization opportunities to the query engine by leveraging vector similarity search. Figure 4 provides an example of the `sem_join` compared to the `sem_sim_join`, where the user specifies the left and right table join keys, and a parameter $K$. The operator performs a left join such that for each row in the left table, the output table will contain $K$ matching rows from the right table with the highest semantically similarity scores.

```
papers_df.sem_join(papers_df, "The paper {abstract:left}
      contradicts the claims in {abstract:right}.")
papers_df.sem_sim_join(papers_df, left_on="abstract",
     right_on="abstract",  K=10)
```

**Figure 4: Example usage of sem_join and sem_sim_join.**

***Sem_topk & Sem_search*** . LOTUS supports a semantic top-k, which takes the langex ranking criteria, as described in Section 2. Programmers can optionally specify a group-by parameter to indicate a subset of columns to group over during ranking, as shown in Figure 5. The groupings are defined using standard equality matches over the group-by columns. Additionally, as the figure shows, LOTUS also provides a top-k variant, `sem_search`, which assumes a semantic similarity-based ranking criteria relative to a natural language query. LOTUS also exposes advanced relevance-based re-ranking functionality for search, allowing users to specify the `n_rerank` parameter during the semantic search. The semantic search in this case will first find the top-$K$ most relevant documents and then re-rank the top-$K$ found documents to return the top `n_rerank`.

***Sem_agg*** performs an aggregation over the input relation, with a langex signature that provides a commutative and associative aggregation function, as shown by Figure 2. Similar to `sem_topk`, LOTUS allows users to specify a group-by parameter.

```
papers_df.sem_topk("the {abstract} makes the most
     outrageous claim", K=10, group_by=[arxiv_domain])
papers_df.sem_search(col="abstract", query="vector
     databases", K=10)
```

**Figure 5: Example usage of sem_topk and sem_search.**

***Sem_group_by*** creates groups over the input dataframe according to the langex grouping criteria and target number of groups. By default, the operator discovers suitable group labels, but the user can also optionally specify target labels. This operator is useful both for unsupervised discovery of semantic groups, and for semantic classification tasks.

***Sem_map & Sem_extract*** both perform a natural language projection over an existing column. While `sem_map` projects to an arbitrary text attribute, `sem_extract` projects each tuple to a list of sub-strings from the source text. This is useful for applications, such as entity extraction, where finding snippets or verified quotes may be preferable to synthesized answers.

## 5 EVALUATION

We systematically evaluate LOTUS on four bulk-semantic processing applications from the AI and data literature: fact-checking, biomedical multi-label classification, search, and topic analysis. Some of these applications can be expressed by recent LLM-based analytics systems, like AI UDFs, UQE and DocETL, allowing us to compare against them; while other applications are not supported by the baseline systems, but LOTUS supports them. For each application, we additionally compare against strong hand-designed pipelines from the literature. We evaluate the LOTUS optimizer, comparing the performance of optimizations for semantic filters, joins, top-k and group-by with our gold algorithms. Lastly, we analyze the statistical accuracy guarantees provided by LOTUS' optimizations. Overall, we find the following:

- Compared to AI-based analytics systems, like AI UDFs, UQE, and DocETL, LOTUS attains similar or up to 170% higher accuracy, while running up to 3.6× faster and offering statistical accuracy guarantees that the other systems do not provide.
- Compared to hand-written pipelines from the AI literature for fact-checking, biomedical classification, and search, LOTUS matches state-of-the-art quality, or exceeds it by up to 100%, in programs involving a few semantic operators.
- The LOTUS optimizer can substantially reduce cost of semantic operator programs, by up to $1,000\times$ compared to high-quality gold algorithms, and provides accuracy guarantees, which hold across repeated trials.

To report controlled latency numbers, we run each baseline and experiment locally on 4 80GB A100 GPUs using Llama-3-70B [6] and E5 embeddings [66], with a batch size of 64 running on vLLM [43], unless otherwise stated. To run DocETL baselines and reproduce some of their benchmarks [61], we additionally use OpenAI [10] models, including GPT-4o-mini-2024-07-18, GPT-4o-2024-08-06 model, and text-embedding-3-small, where noted. For our experiments that use OpenAI models , we control for cost by limiting execution to 64-way thread parallelism. For reproducibility, we set temperature to $t = 0$ for all methods and baselines, unless otherwise stated.

*5.0.1 Benchmarked Methods.* We briefly overview the main methods we benchmark and tested parameters.

***AI UDFs.*** Many database vendors [1, 8, 9, 14, 53, 67] now support AI UDFs, which include map-like, row-wise LLM operations and primitives for vector search. To control for serving infrastructure, we implement these programs using LOTUS running on vLLM, and restrict our API to `sem_map`, `sem_search` and `sem_sim_join`.

***UQE.*** UQE [25] proposes an optimized LLM-powered filter method, which is relevant to this work. UQE also optimizes non-LLM aggregates (e.g. COUNT) integrated with LLM-based queries, but this is beyond the scope of our evaluation. Since the code is not open-source, we implement the UQE filter in about 100 line of code in python. The UQE filter method takes an LLM call budget and provides a best-effort embedding-based approximation. In contrast, LOTUS provides a general proxy-oracle approximation method, where the proxy may be embedding-based or LLM-based. To provide a fair comparison between UQE and LOTUS, we normalize the *latency budget* when comparing against LOTUS programs that use LLM-based proxies, and we normalize *the LLM call budget* when comparing against LOTUS programs that use embedding-based proxies. We also hyper-parameter tune the mini-batch size used for UQE's active learning algorithm.

***DocETL.*** DocETL [61] uses an LLM agent as the query optimizer to perform rewrites to programs, allowing programs to specify separate LLMs for the optimizer and execution. Our experiments find that the optimizer often fails, and among runs where the optimizer succeeds, performance varies. Based on personal communication with the authors, we run the DocETL baselines multiple times. We obtain three successful runs and report the average performance of these three successes. We also try using both Llama-70B as the DocETL optimizer, following our setup for each other baseline, and GPT-4o-mini as the DocETL optimizer, following the DocETL preprint [61]. However, we find both fail to produce a successful run in Section 5.2. After reaching out to the authors, we confirm a mistake in the pre-print and find that GPT-4o is required for the DocETL optimizer for our evaluation on BioDEX in Section 5.2. We also note we found several bugs in the DocETL codebase, which we have done our best to fix and have also coordinated with the authors about. Since the codebase is an evolving artifact, we report the commit number we used for our evaluation[6].

***LOTUS.*** We set default accuracy targets of $\gamma = 0.9$ and failure probabilities of $\delta = 0.2$. We provide a detailed ablation, varying these parameters in Section 5.5. Additionally, our sampling size uses 0.01% of the data, similar to other works [61], and a minimum sample size of $s = 100$ for smaller datasets, following prior work [37].

## 5.1 Fact-Checking

Fact-checking systems determine whether a given claim is correct, typically based on verifiability with a knowledge corpus. FacTool [23] is a recent open-source research work that provides a multi-step fact-checking pipeline, involving claim extraction, query generation, tool querying, evidence collection, and verification. We consider the task of using constructing the FacTool pipeline, which was originally written in over 750 lines of code. We use the FEVER [64] dataset, a claim verification dataset, for our evaluation. Claim verification is based on a corpus of 5.5 million Wikipedia articles, and each claim is labeled with one of three gold labels, "Supported", "Refuted", or "NotEnoughInfo". We merge the latter two labels into a single class, "Not Supported", following prior work [23] for our evaluation. To control for our API spending budget, we sample 1,000 claims from the development dataset.

**Table 2: Fact-checking Performance on the FEVER Dataset.**

| Method | Accuracy | ET (s), with batching | ET (s), no batching | LoC |
|---|---|---|---|---|
| FacTool | 80.9 | N/A | 5396.11 | > 750 |
| AI UDF: map, search, map | 89.9 | 688.9 | 4,454.2 | < 50 |
| AI UDF map, search + UQE filter | 66.0 | 184.4 | 738.3 | 150 |
| LOTUS map, search, filter (unopt.) | 91.2 | 329.1 | 989.0 | < 50 |
| LOTUS map, search, filter (opt.) | 91.0 | 190.0 | 776.37 | < 50 |

***Baselines.*** For all baselines we use ColBERT [42] as the retreiver model[7] and Llama-70B served with vLLM. We run FactTool's open source codebase [5] to measure its performance. The AI UDF baseline uses a simple semantic map-search-map dataflow, following FacTool's pipeline. First each claim is mapped to two search queries. Then the generated queries are used for search over the Wikipedia corpus. Lastly, each claim appended with retrieved context is mapped to truth label, along with a reasoning. We use the same prompts found in FacTool [5], which include 3 demonstrations for generating search queries in the first semantic mapping, and chain-of-thought prompting in the second semantic mapping. The UQE baseline follows the first two steps of the AI UDF baseline, but replaces the final step with UQE's embedding-based LLM-powered filter. Our LOTUS program similarly follows a map-search-filter pipeline and uses a Llama-8B proxy model. To provide a fair comparison, we tune the UQE LLM call budget to normalize execution time with the optimized LOTUS program. Since DocETL does not provide search primitives, we are unable to benchmark it in this application.

***Results.*** We report the accuracy of each method, an estimate of lines of code (LoC), and average execution time over 10 runs in seconds, both with and without batching to provide a fair comparison with FacTool, which provides a sequential implementation only. Overall, Table 2 demonstrates that the optimized LOTUS program reproduces state-of-the art accuracy, comparable to that of FacTool, with 7× faster unbatched execution and 28× faster batched execution, and in less than 50 lines of user code, while also outperforming each baseline.

Comparing the optimized LOTUS program to the AI UDF baseline, we see that both acheive comparable accuracy in relatively few lines of code. However, the LOTUS `sem_filter` can be more heavily optimized compared to the generic AI UDF map operation. This allows the LOTUS program to attain 3.6× faster execution, attributable to the `sem_filter`'s short LLM generations, which leads to cheaper decoding, and our proxy-based optimization for

[6] https://github.com/ucbepic/docetl/commit/93050998077a9eb6fc1ee99dc96d1e0a222a987f

[7] FactTool's pipeline, by default, performs retrieval with a Google Search API [3]. We evaluate the pipeline with both the default retrieval API, and with the open-source ColBERT [42] index. We find that the results are similar, and we report the results using ColBERT for retrieval to hold the retriever model constant with the other baselines.

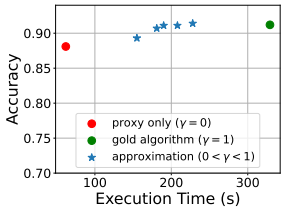


**Figure 6: Accuracy vs. execution time (s) for the LOTUS fact-checking program on the FEVER dataset. We compare the performance using the proxy only (red circle) or oracle only (green circle) for filters to the pipeline implemented using our filter approximation with both models for varied the precision and recall target, at $\delta = 0.2$ (stars).**

filters. We also observe that the LOTUS program outperforms the UQE program by 38% accuracy at an equivalent latency budget. While our filter optimization adaptively learns when to leverage the proxy in order to meet an accuracy target, UQE's embedding-based approximation provides no accuracy guarantees.

Finally, we compare the optimized LOTUS program to the un-optimized LOTUS program. We observe that the optimized LOTUS program achieves 99.8% accuracy relative to the un-optimized one, while reducing batched execution time by 1.7×. Figure 6 highlights the diverse performance trade-offs achievable by our `sem_filter` approximation. The plot shows the accuracy and execution time of the LOTUS program using the gold algorithm with Llama 70B, or using the proxy only, shown respectively by the green and red circles. By varying the filter recall and precision target, our cascade-based approximation offers diverse trade-offs between accuracy and execution time, shown by the stars.

## 5.2 Biomedical Multi-label Classification

Biomedical classification entails processing complex, lengthy patient documents. We consider the extreme multi-label classification task on the BioDEX Dataset [29], which consists of a corpus of 65,000 biomedical articles, and ground-truth expert-created drug safety reports constructed from each article. The task is to classify the patient's drug reactions, given each medical article. Notably, there are 24,000 possible drug-reaction labels, making this task an extreme multi-label classification task. Due to the large number of possible labels, leveraging an LLM to perform inference is difficult, and this setting has been studied in prior works [28]. We show below that this task can be efficiently modeled and optimized using the semantic join. Similar to prior work [29] we sample 250 patient articles for our evaluation to control for our API spending budget.

***Baselines.*** The search baseline performs batched similarity search over each patient article and the set of reaction labels. For the baseline LLM-based analytics systems, we implement a LLM-powered join over the patient articles and set of candidate reaction labels, followed by a ranking step, if supported. The AI UDF program entails a naive, nested-loop join pattern using row-wise LLM operators; however we note this is prohibitively costly and we report estimated latency. While UQE does not explicitly study joins, we implement a UQE join using its optimized filter. The DocETL program uses a join and reduce, which uses listwise ranking, following the preprint [61]. We note that we are unable to successfully run DocETL using Llama-70B or GPT-4o-mini for its optimizer so results (Table 3, 4) are shown using GPT-4o for the DocETL optimizer. LOTUS supports both join and ranking. Although LOTUS provides a `sem_topk` operator, we limit our ranking step to list-wise ranking to maintain a fair comparison with DocETL, and revisit ranking in Section 5.3. Our main results are shown in Table 3, using Llama-70B and E5 embeddings and sampling size of 0.01%. We also provide an additional set of benchmarks in Table 4 comparing LOTUS and DocETL following the models, prompts and parameters used in the original DocETL preprint. The DocETL setup uses GPT-4o-mini, text-embedding-3-small embeddings and a sampling size of 500. Once again, DocETL fails when we attempt to use GPT-4o-mini for the optimizer, so we use GPT-4o. DocETL's join takes an LLM call budget, which we control to compare against the other baselines.

**Table 3: Biomedical Multi-label Classification Results on the BioDEX Dataset with Llama-70b**

| Method | RP@5 | RP@10 | Execution Time (s) | # LM Calls |
|---|---|---|---|---|
| Search | 0.106 | 0.120 | 2.91 | 0.00 |
| AI UDF * | N/A | N/A | 2,144,560 | 6,092,500 |
| UQE | 0.115 | 0.114 | 6,559 | 15,000 |
| DocETL Join (avg of 3 successes)** | 0.180 | 0.219 | 2050 | 13,185 |
| DocETL Join + Rank (avg of 3 successes)** | 0.262 | 0.282 | 2,342 | 13,433 |
| LOTUS Join | 0.212 | 0.213 | 2,340 | 5,644 |
| LOTUS Join + Rank | 0.265 | 0.280 | 2,503 | 5,869 |

**Table 4: Additional Comparison Following DocETL's Original Benchmarks on the BioDEX Dataset with GPT-4o-mini**

| Method | RP@5 | RP@10 | Execution Time (s) | # LM Calls |
|---|---|---|---|---|
| DocETL Join + Rank (avg of 3 successes)** | 0.268 | 0.287 | 583 | 35,669 |
| LOTUS Join + Rank | 0.289 | 0.296 | 647 | 32,026 |

* AI UDF is infeasible to run and latency is estimated under linear-scaling assumption in number of batched LLM calls.

** The DocETL baselines requires using GPT-4o as its optimizer and still exhibit frequent failures, with, on average, 1 in 3 runs failing. We rerun the system multiple times and report results averaged only over 3 successful runs.

***Results.*** Table 3 demonstrates the LOTUS programs consistently match or exceed the accuracy of all baselines, while also providing performance guarantees, unlike DocETL, which requires multiple reruns and manual selection of the optimizer LLM to produce high-quality results. The table reports the rank-precision@5 (RP@5) and rank-precision@10 (RP@10), following prior work [28], as well as execution time in seconds and the number of LLM calls. The LOTUS join with ranking program achieves higher rank-precision than the standalone LOTUS join program, and we show both.

We begin by informally comparing these accuracy results to that of D'Oosterlinck et al. [28], a state-of-the art AI pipeline that composed a multi-step DSPy [41] program compiled using a combination of Llama-2-7b-chat, GPT-3.5-turbo, and GPT-4-turbo. D'Oosterlinck et al. report 24.73 RP@5 and 27.67 RP@10 for the compiled program, which is comparable to result quality achieved by the simple LOTUS join with ranking program.

Turning our attention to the baselines measured in Table 3, we see that, first, the search baseline offers low results quality, with the LOTUS programs achieving over 2× higher RP@5 and RP@10.